

SIMATS ENGINEERING  
ASSESSMENT TOOL 1

**COURSE CODE: CSA14**

**COURSE NAME: Compiler Design**

---

**COURSE OUTCOME COVERED**

***CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)***

*Assessment Tool Weightage: 40%*

---

**QUESTIONS: 5**

**Total Marks:50**

**Mode: In Class**

**Hours : 60 mts**

Annexure A

**ANALYTICAL PROBLEM SOLVING**

---

***Code of Conduct:***

*I M Harinath/192425020 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

| Q. No                 | Problem Understanding (20) | Method Selection (20) | Solution Process (25) | Accuracy of Calculation (20) | Interpretation of Results (15) | Total Score (out of 100) |
|-----------------------|----------------------------|-----------------------|-----------------------|------------------------------|--------------------------------|--------------------------|
| 1                     | / 4                        | / 4                   | / 4                   | / 4                          | / 4                            |                          |
| 2                     | / 4                        | / 4                   | / 4                   | / 4                          | / 4                            |                          |
| 3                     | / 4                        | / 4                   | / 4                   | / 4                          | / 4                            |                          |
| 4                     | / 4                        | / 4                   | / 4                   | / 4                          | / 4                            |                          |
| 5                     | / 4                        | / 4                   | / 4                   | / 4                          | / 4                            |                          |
| OVERALL RUBRICS VALUE |                            |                       |                       |                              |                                |                          |
|                       | / 4                        | / 4                   | / 4                   | / 4                          | / 4                            |                          |
|                       | / 20                       | / 20                  | / 25                  | / 20                         | / 15                           | / 100                    |

# COI - ATI

Q. Expression:  $y = (p + a) * r - s / t$

Lexical Analysis:

Lexeme

Token

y

Identifier

=

Assignment operator

(

Left Parenthesis

p

Identifier

+

Addition operation

a

Identifier

)

Right Parenthesis

\*

Multiplication operator

r

Identifier

-

Sub operator

s

Identifier

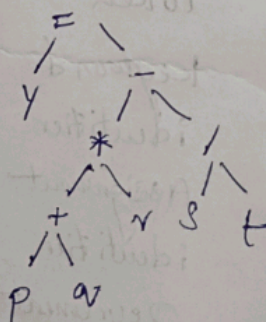
/

Division operator

t

Identifier

b) Syntax Tree



c) Intermediate Code

$$t_1 = p + a$$

$$t_2 = t_1 * r$$

$$t_3 = s / t$$

$$t_4 = t_2 - t_3$$

$$y = t_4$$



(a) optimization

$$t_1 = p + w$$

$$t_2 = t_1 * r$$

$$t_3 = s / t$$

$$y = t_2 - t_3$$

(c) Target Code (example)

LOAD P

ADD w

MUL ~~r~~

STORE R<sub>1</sub>

LOAD s

DIV t

STORE R<sub>2</sub>

LOAD R<sub>1</sub>

SUB R<sub>2</sub>

STORE y

Result: The Compiler Converts the source program into efficient machine instructions.

(2)

total = value -- + + + + + Count -- \* 10.5;

Lexeme

float

total

=

value

--

++

Count

--

\*

10.5

;

Token

keyword

identifier

Assignment

identifier

Decrement

increment

Identifier

Decrement

MUL

Floating Constant

Delimiter



## Conflict Resolution

Lex always matches the longest possible token

Ex:-

++ → INC

-- → DEC

instead of two '+' or '-' symbols

Conclusion:

Proper lexical rules remove ambiguity and correctly identify operators and identifiers.

(3)

```
int main()  
{  
    int i, j, sum = 0;  
    for (i = 0; i < 3; i++) {  
        for (j = 0; j < 2; j++) {  
            sum = sum + i * j;  
        }  
        printf("%d\n", sum);  
    }  
    return 0;  
}
```

Token Count

| Statement                           | tokens |
|-------------------------------------|--------|
| #include <stdio.h>                  | 2      |
| int main() {                        | 5      |
| int i, j, sum = 0;                  | 8      |
| <del>int i;</del><br>first for loop | 14     |
| sum = sum + i * j;                  | 8      |
| printf("%d\n", sum);                | 6      |
| return 0;                           | 3      |
| closing braces                      | 3      |

Total tokens = 63



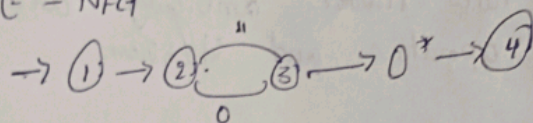
Note: Comments and white spaces are ignored by the lexical analyzer.

- ④ Construct a non-deterministic finite automata with  $\epsilon$ -transitions to represent the language denoted by the regular expressions  $10+(0+1)^*0.1$ .

Regular expression

$10+(0+1)^*0.1$

$\epsilon$ -NFA



Explanation:-

- 10 represents one possible path
- $(0+1)^*$  allows zero or more repetitions of either 0 or 1
- $0^*$  accepts zero or more zeros.
- Final 1 leads to the accepting state.

- ⑤ Construct the language for the following regular expressions.

a)  $(\epsilon+a)^*$

Language:  $\{\epsilon, a, aa, aaa, aaaa, \dots\}$

b)  $(a+b)^*aa$

Strings over  $\{a,b\}$  ending with  $aa$

$aa$   
 $baa$   
 $abaa$   
 $bbaa$



c)  $aba$

Exactly one  $b$  with any number of  $a$ 's before and after

Ex:-  
 $bab$   
 $ba$   
 $aab$   
 $baa$   
 $aaabaaa$

d)  $(ab+ba)^*$

Language: all string formed by repeating either "a" or "ab" any number of times (including zero times)

Examples:

$L = \{ \epsilon, a, ab, aa, aab, aba, abab, abaa, \dots \}$

e)  $(ab)^* + (ba)^*$

Language: union of strings formed by repeating "ab" or repeating "ba" zero or more times.

Examples:

$L = \{ \epsilon, ab, abab, ababab, ba, babab, \dots \}$